`DbContextOptionsBuilder` instance. This builder can set a host of internal properties of EF Core and lets you replace all the internal services of EF Core if you want.

The configuration for your app is, again, nice and simple, as you can see in the following listing. You set the database provider with the `UseSqlite` extension method, made available by the Microsoft.EntityFrameworkCore.SQLite package, and pass it a connection string.

Listing 12.4 Registering a `DbContext` with the DI container

```
using Microsoft.EntityFrameworkCore;
WebApplicationBuillder builder = WebApplication.CreateBuilder(args);
var connString = builder.Configuration                    ❶
        .GetConnectionString("DefaultConnection");        ❶

Builder.Services.AddDbContext<AppDbContext>(              ❷
        options => options.UseSqlite(connString));        ❸

WebApplication app = builder.Build();
app.Run();
```

❶ The connection string is taken from configuration, from the ConnectionStrings section.

❷ Registers your app's DbContext by using it as the generic parameter

❸ Specifies the database provider in the customization options for the DbContext.

NOTE If you're using a different database provider, such as a provider for SQL Server, you need to call the appropriate `Use*` method on the `options` object when registering your `AppDbContext`.

The connection string is a typical secret, as I discussed in chapter 10, so loading it from configuration makes sense. At runtime the correct configuration string for your current environment is used, so you can use different databases when developing locally and in production.